Topic: Operations on Doubly Linked List

Q. Discuss the various operations (Traversal, Insertion, Deletion, Searching) on a Doubly Linked List. How does the presence of a 'previous' pointer change these operations compared to a Singly Linked List?

> **Definition to Remember**

> A **D
(previo
deletio

Operations on Doubly Linked List

- **Forward Traversal:** Start at `head`, loop using `temp = temp->next`.

- **B

2. Insertion Operations

In a DLL, both `next` and `prev` pointers must be carefully updated.

| Position | Logic Steps |

| :--- | :

3. Deletion Operations

Deleting is highly efficient (O(1)) if a pointer to the target node (`del_node`) is already known, because we don't need to traverse from the head to find its preceding node.

Logic to delete `del_node`:

1. If de

4. Advantages vs Disadvantages

| | Detail |

| :--- | :

> **Must-Write Points (for 10 marks)**

> 1. D

> **Quick Recall**

> `Dou
memo

Complete Executable C Code: Doubly Linked List Full Implementation

```c
#inclu

struct Node {
int dat

// Insert at Head - O(1)
struct

if (head != NULL) head->prev = newNode;
return

// Forward & Backward Traversal
void tr

void traverseBackward(struct Node *head) {
if (hea
```

```
printf("Backward List: ");
                                                                    while (

int main() {
                                                                    struct

head = insertHead(head, 30);
                                                                    head =

traverseForward(head);
                                                                    travers

return 0;
                                                                    }
```